

Búsqueda de materiales necesarios para la practica 22 Enero

En base a la investigación realizada se buscará realizar el proyecto con los siguientes materiales:

Coche prototipo: Permite simular el comportamiento de un vehículo real a escala.

Raspberry pi 5: Pieza central del proyecto donde correrá el programa.

Módulo de cámara para raspberry pi: Indispensable para la detección de objetos.

Buzzer: Optimo para poner en alerta al usuario.

Tarjeta micro sd: Necesaria para que la raspberry cuente con almacenamiento.

Baterías recargables: Necesarias para el funcionamiento del vehículo prototipo.

Fuente de alimentación para raspberry: Indispensable para el funcionamiento.

Carcasa para raspberry: No es indispensable sin embargo se incluirá para la protección y refrigeración de esta.

Breve investigación teórica sobre el algoritmo YOLO 26 Enero

Durante esta investigación aprendí el funcionamiento del algoritmo YOLO: la imagen es dividida en partes mas pequeñas el algoritmo hace pasadas por estas imágenes y busca el objeto u objetos que deseamos encontrar. Funcionamiento explicado en la siguiente matriz.

y =	P_r	Valores entre 0 y 1, cuanto más se acerca a 1 mas probable es que se haya detectado el objeto deseado
	B_x	Estos valores indican el centro del bounding box en la imagen
	B_y	
	B_w	Estos valores indican el tamaño que abarca el bounding box
	B_h	
	C_1	Clase de objeto que deseamos detectar (1 Si se detectó, 0 Si no)
	C_2	Clase de objeto que deseamos detectar (1 Si se detectó, 0 Si no)

En caso de que P_r sea igual a 0 (o un valor establecido por el usuario) el algoritmo no se ejecuta

Intersección sobre unión (Intersection over unión, IoU): El bounding box no siempre aparecerá donde se desea, y tampoco abarcara correctamente todo el objeto que queramos detectar, este se basa en valores donde 0 es igual a una pésima predicción y 1 es igual a una predicción perfecta

Supresión de no-máximos (Non-Max Suppression): En la practica el modelo puede detectar por cada objeto múltiples bounding boxes.

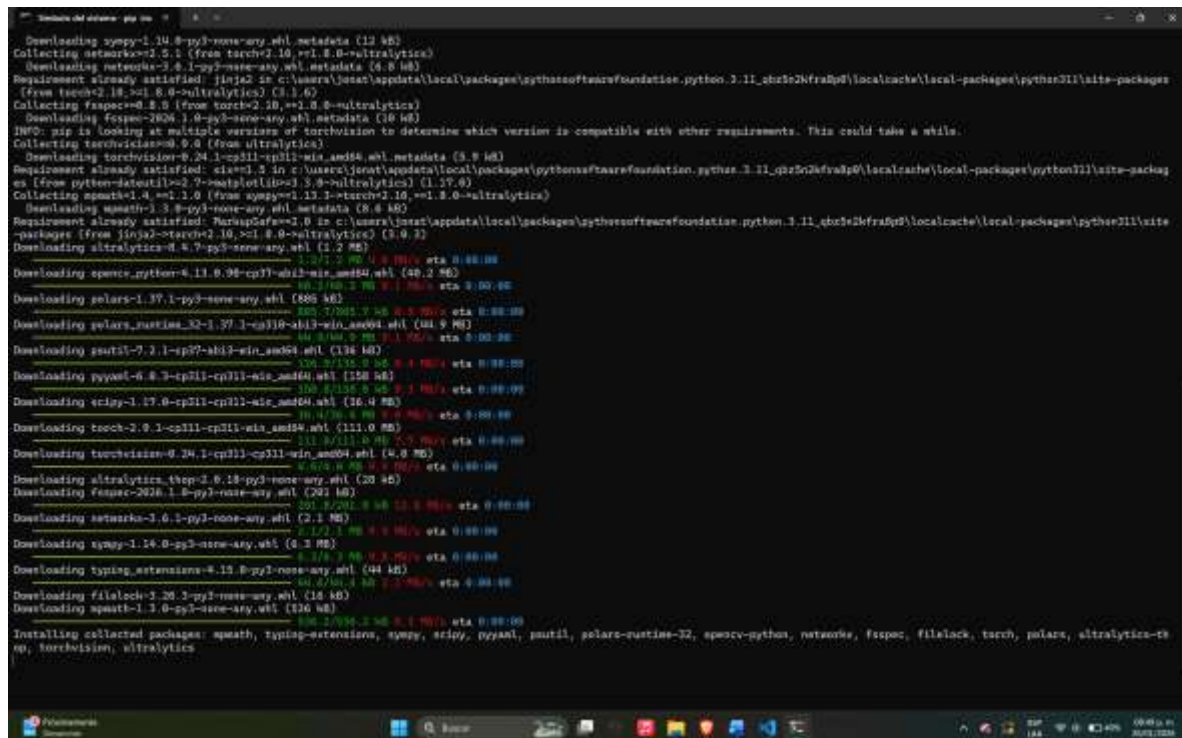
Proceso de limpieza de bounding boxes

- 1 Elimina los bounding boxes con $Pr < \text{Umbral de confianza (0.6)}$
- 2 Tomar el bounding box con el P_r más alto como predicción
- 3 Calcular el IoU entre el bounding box seleccionado y los bounding boxes restantes
- 4 Descartar los bounding boxes con $\text{IoU} > \text{Umbral (0.5)}$
- 5 Repetir los pasos de 2 a 4 con los bounding boxes restantes
- 6 Detenerse cuando no haya más bounding boxes por limpiar

Referencia: <https://www.youtube.com>

Investigación practica sobre el funcionamiento de YOLO 26 Enero

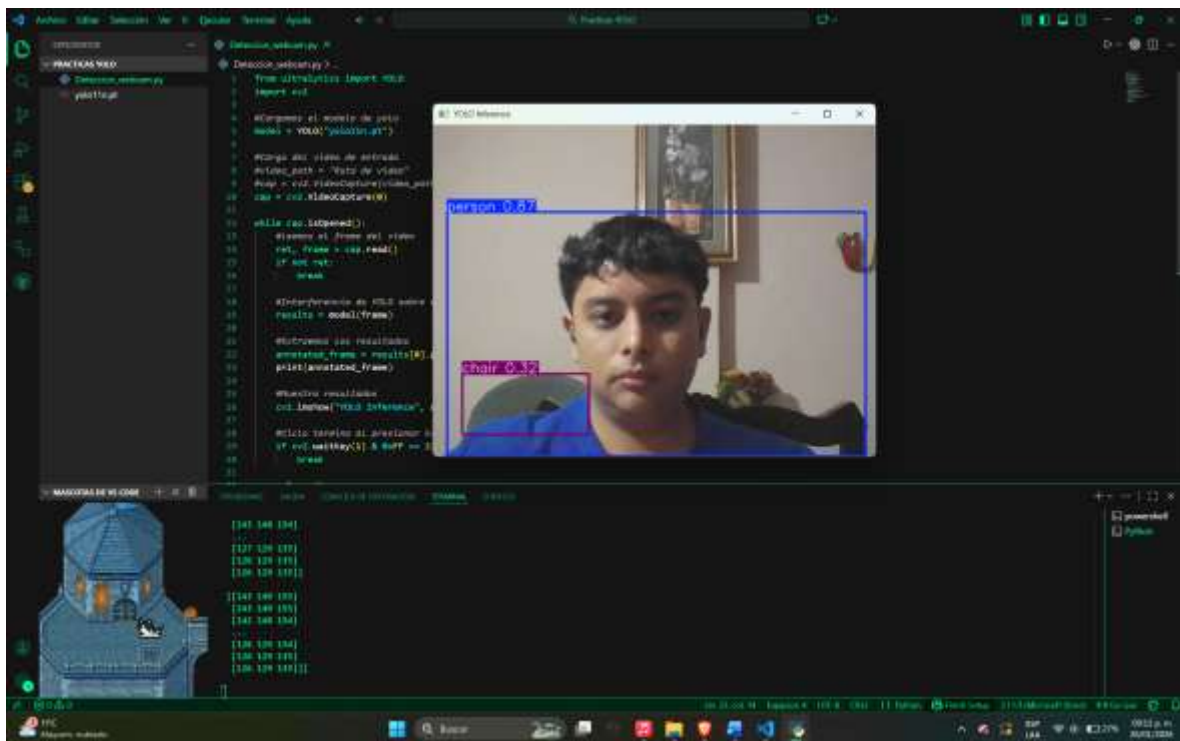
Instalación de la librería ultralytics: `pip install ultralytics`



```
Downloading numpy-1.14.0-py3-none-any.whl.metadata (12 kB)
Collecting networkx<2.5.1, from torch>2.1.0, <=1.8.0->ultralytics)
Downloading networkx-2.4.1-py3-none-any.whl.metadata (6.8 kB)
Requirement already satisfied: Jinja2 in c:\users\joser\appdata\local\packages\pythonsoftwarefoundation.python.3.11.qbz5n2kfra8p0\localcache\local-packages\python311\site-packages
(from torch>2.1.0, <=1.8.0->ultralytics) (3.1.6)
Collecting fsspec<=2020.1.0-py3-none-any.whl.metadata (10 kB)
INFO: pip is looking at multiple versions of torchvision to determine which version is compatible with other requirements. This could take a while.
Collecting torchvision<0.9.0, from ultralytics)
Downloading torchvision-0.14.1-cp311-cp311-macosx_10_9_arm64.whl.metadata (5.9 kB)
Requirement already satisfied: Pillow in c:\users\joser\appdata\local\packages\pythonsoftwarefoundation.python.3.11.qbz5n2kfra8p0\localcache\local-packages\python311\site-packages
(from torchvision>0.2.0, <=0.10.0->ultralytics) (10.4.0)
Collecting spack<1.4, <=1.2.0, from numpy>1.13.1, <=1.8.0->ultralytics)
Downloading spack-1.3.0-py3-none-any.whl.metadata (8.8 kB)
Requirement already satisfied: MarkupSafe in c:\users\joser\appdata\local\packages\pythonsoftwarefoundation.python.3.11.qbz5n2kfra8p0\localcache\local-packages\python311\site-packages
(from Jinja2>2.10.1, <=1.8.0->ultralytics) (2.0.1)
Downloading ultralytics-0.0.1-py3-none-any.whl (11.2 MB)
100% |#####| 11.2 MB 0.0 MB/s eta 0:00:00
Downloading opencv-python-4.11.0.96-cp37-cp37m-macosx_10_9_arm64.whl (48.2 MB)
100% |#####| 48.2 MB 0.1 MB/s eta 0:00:00
Downloading polars-1.37.1-py3-none-any.whl (885 kB)
100% |#####| 885 kB 0.0 MB/s eta 0:00:00
Downloading polars_polars_32-1.37.1-cp310-cp310-macosx_10_9_arm64.whl (64.9 MB)
100% |#####| 64.9 MB 0.1 MB/s eta 0:00:00
Downloading pauli-7.1.1-py37-abi3-macosx_10_9_arm64.whl (136 kB)
100% |#####| 136 kB 0.0 MB/s eta 0:00:00
Downloading pyyaml-6.0.1-cp311-cp311-macosx_10_9_arm64.whl (128 kB)
100% |#####| 128 kB 0.0 MB/s eta 0:00:00
Downloading scipy-1.17.0-cp311-cp311-macosx_10_9_arm64.whl (36.4 MB)
100% |#####| 36.4 MB 0.0 MB/s eta 0:00:00
Downloading torch-2.9.1-cp311-cp311-macosx_10_9_arm64.whl (111.0 MB)
100% |#####| 111.0 MB 0.0 MB/s eta 0:00:00
Downloading torchvision-0.14.1-cp311-cp311-macosx_10_9_arm64.whl (14.9 MB)
100% |#####| 14.9 MB 0.0 MB/s eta 0:00:00
Downloading ultralytics_thop-2.0.18-py3-none-any.whl (20 kB)
100% |#####| 20 kB 0.0 MB/s eta 0:00:00
Downloading fsspec-2020.1.0-py3-none-any.whl (201 kB)
100% |#####| 201 kB 0.0 MB/s eta 0:00:00
Downloading networkx-2.4.1-py3-none-any.whl (2.1 MB)
100% |#####| 2.1 MB 0.0 MB/s eta 0:00:00
Downloading numpy-1.14.0-py3-none-any.whl (6.3 MB)
100% |#####| 6.3 MB 0.0 MB/s eta 0:00:00
Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
100% |#####| 44 kB 0.0 MB/s eta 0:00:00
Downloading filack-2.20.3-py3-none-any.whl (15 kB)
100% |#####| 15 kB 0.0 MB/s eta 0:00:00
Downloading spack-1.3.0-py3-none-any.whl (136 kB)
100% |#####| 136 kB 0.0 MB/s eta 0:00:00
Installing collected packages: spack, typing-extensions, numpy, scipy, pyyaml, pauli, polars-polars-32, opencv-python, networkx, fsspec, filack, torch, polars, ultralytics-thop, torchvision, ultralytics
```

Código fuente: <https://github.com/Trolhappy>

Resultados obtenidos: El algoritmo parece funcionar satisfactoriamente, sin embargo, debe ser probado en una variedad mas amplia de entornos y con diversos objetos para confirmar su fiabilidad.

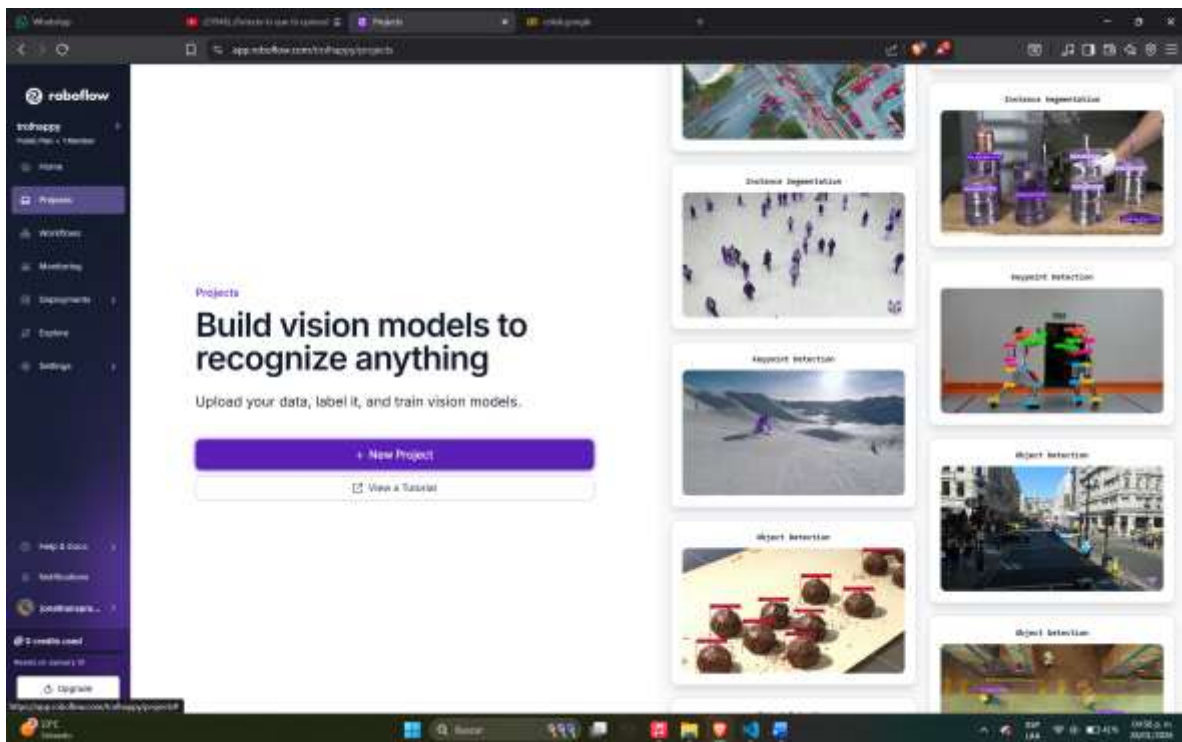


Referencia: <https://www.youtube.com>

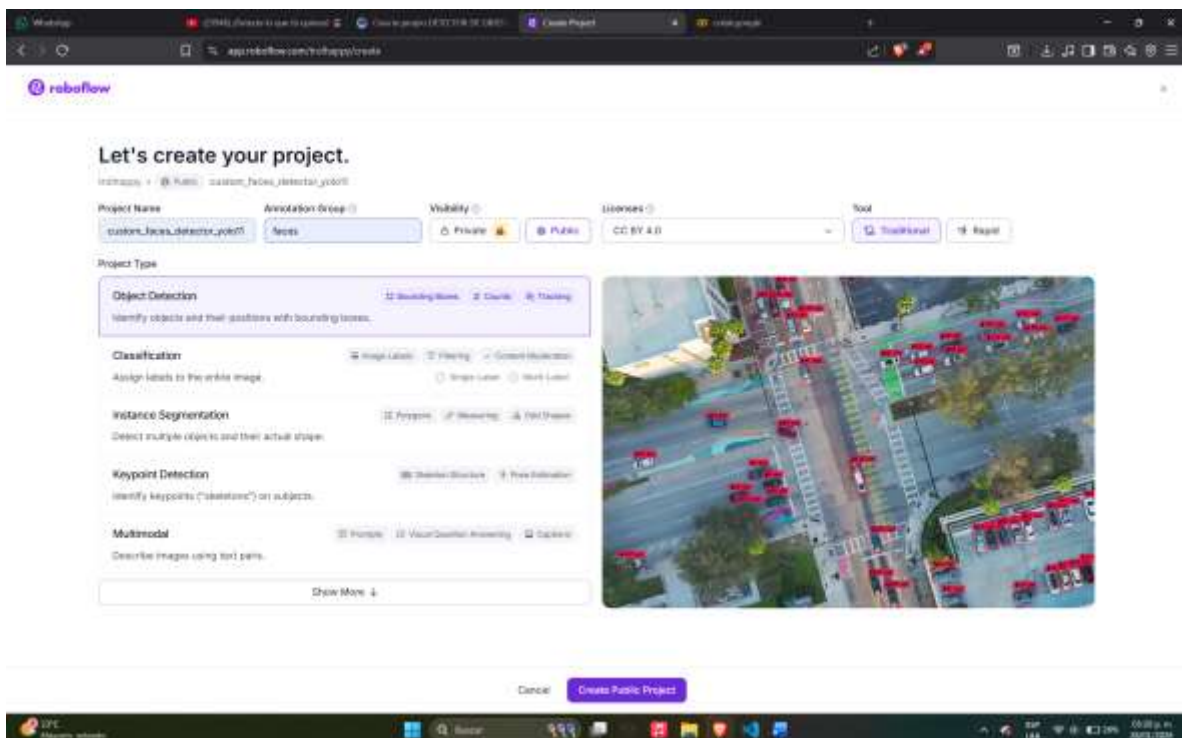
Como entrenar el algoritmo YOLO 26 Enero

Roboflow nos permite crear datasets para entrenar nuestro modelo de detección de objetos mediante imágenes, esta herramienta nos da múltiples opciones desde encontrar objetos automáticamente lo cual nos ayuda a reducir de manera considerable el tiempo, también podemos etiquetar los objetos de forma manual para verificar que las etiquetas sean colocadas correctamente.

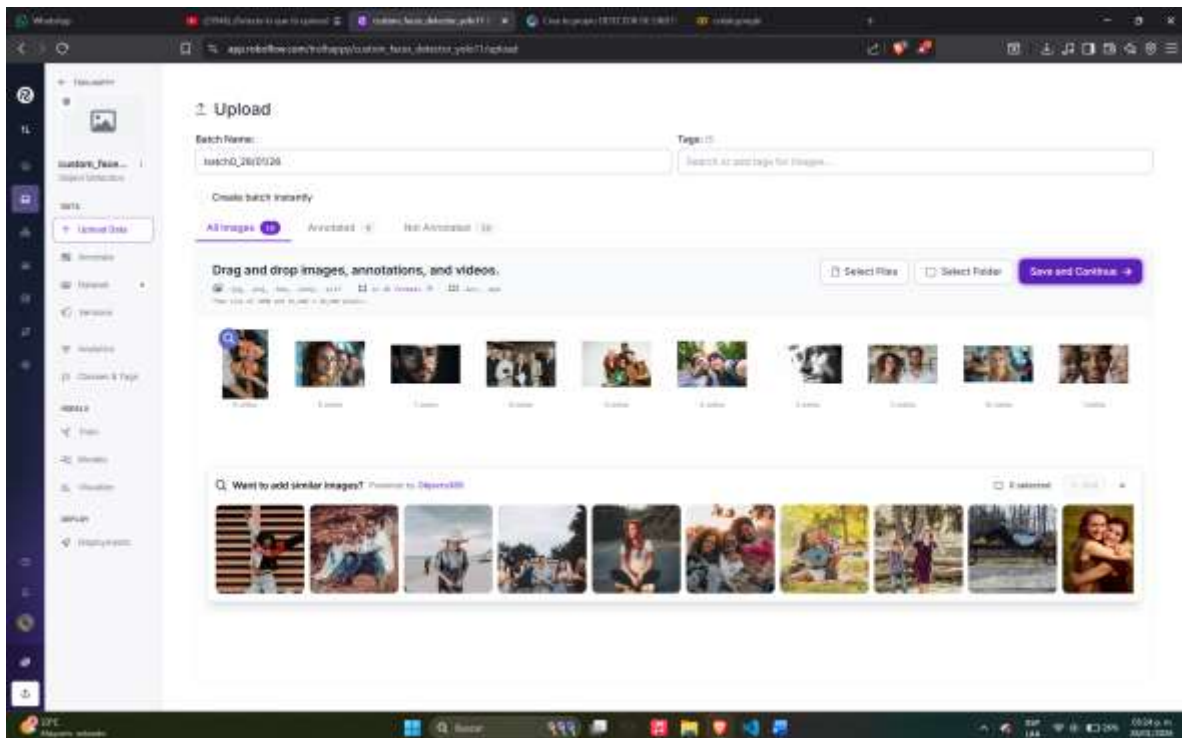
Creación del proyecto:



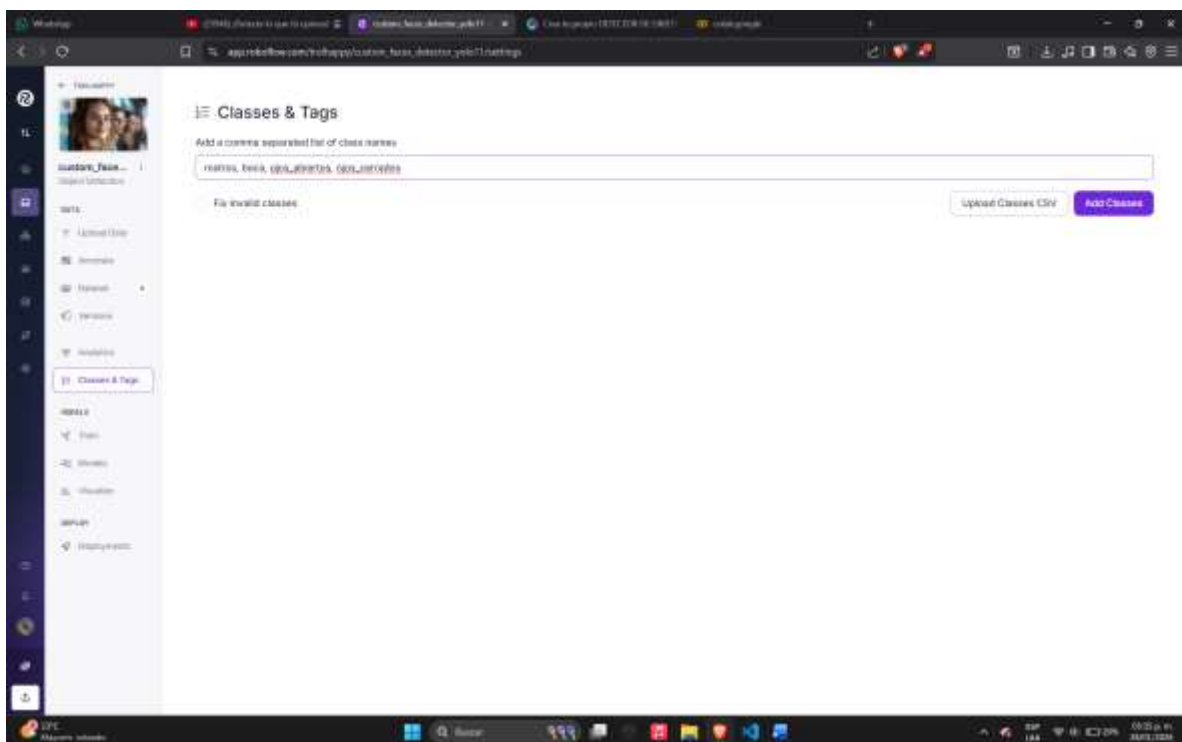
Le damos nombre al proyecto, annotation group es una referencia de lo que trata el dataset, en tool seleccionamos tradicional, en project type mantenemos object detection.



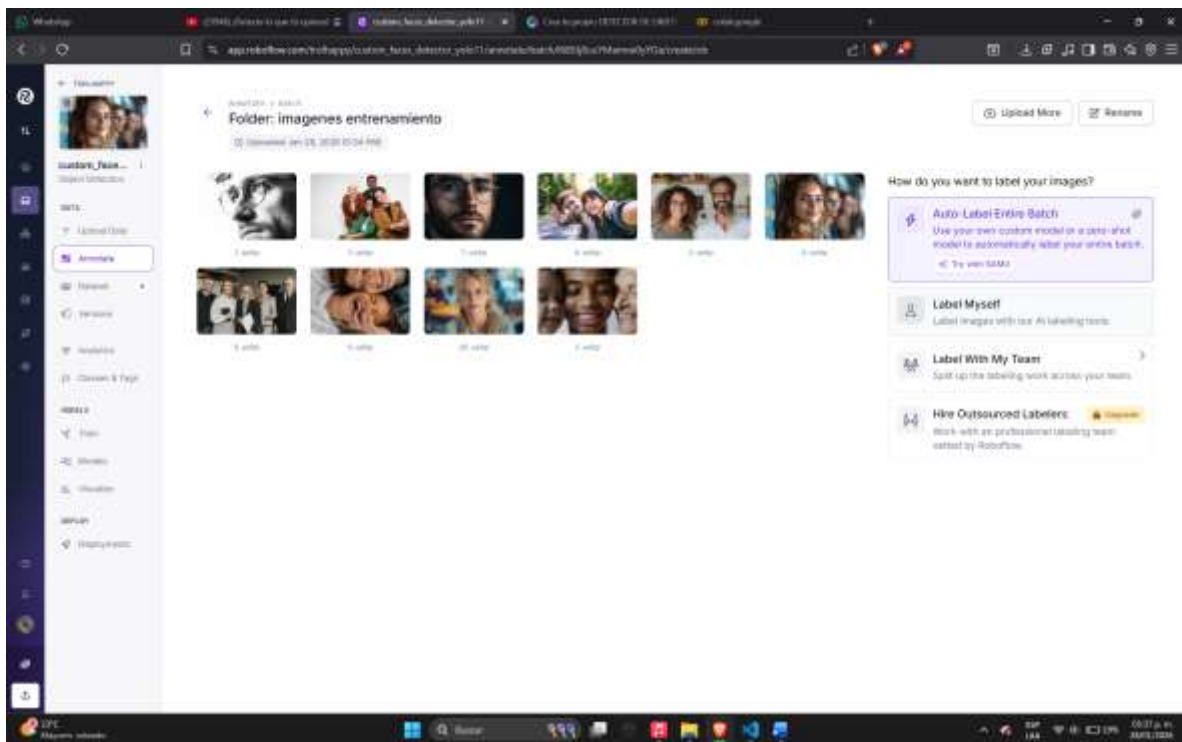
Cargamos las imágenes para el entrenamiento, asignamos un nombre, y damos en guardar y continuar.



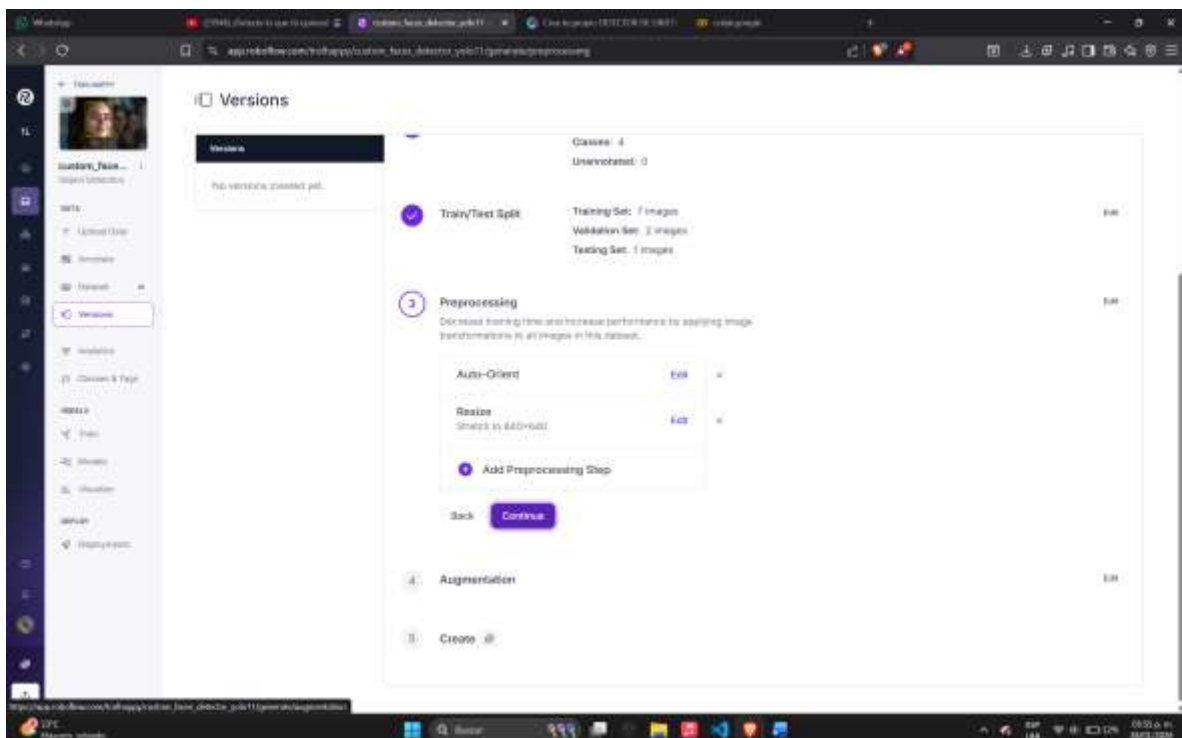
Añadimos las clases que queremos detectar



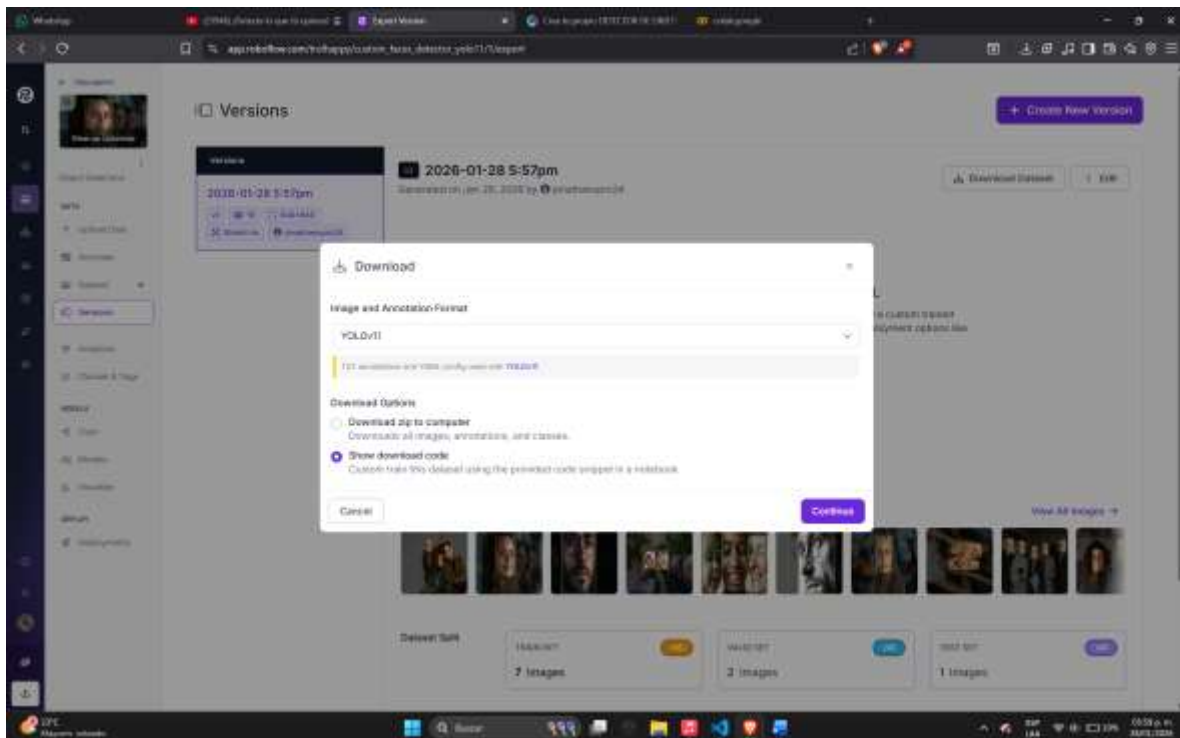
Etiquetamos los objetos a detectar



Preprocesamos las imágenes en 640x640 ya que es la entrada que usa YOLO, en aumentativo podemos añadir más imágenes similares si así lo queremos, después damos click en crear



Seleccionamos el formato YOLOv11 y seleccionamos que nos muestre el código para descargarlo.



Usamos Google Colab para continuar

En la primera celda de código: simplemente instalamos la librería de roboflow

En la segunda celda de código: pegamos los datos que nos dio roboflow al descargar nuestro dataset

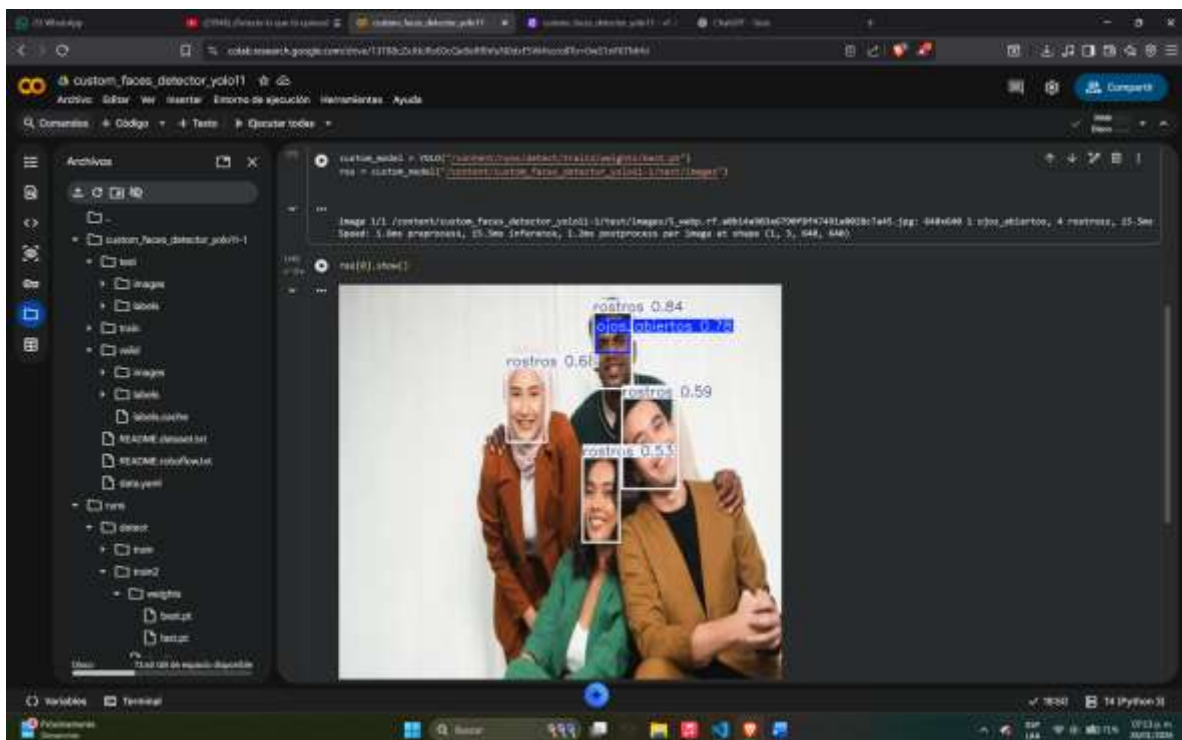
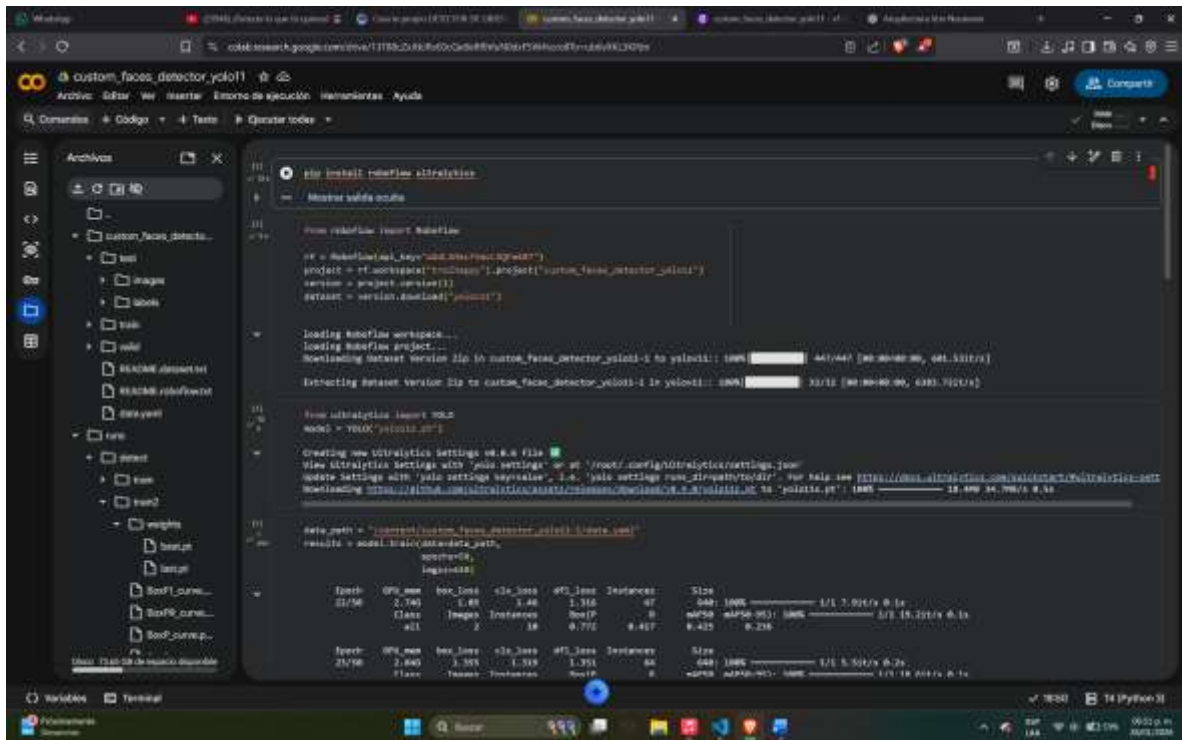
En la tercera celda de código: importamos el modelo yolo11s

En la cuarta celda de código: copiamos la ruta del archivo data.yml y en epochs son las veces que el modelo recorre todo el conjunto de entrenamiento para el proceso de aprendizaje.

Al final de este paso la celda nos mostrara una ruta donde se generaran 2 archivos best.pt para el modelo con mejor predicción y last.pt que es el ultimo modelo creado. Aquí debemos jugar con los epochs para obtener el mejor rendimiento, también es importante el dataset, entre más imágenes para entrenar nuestro modelo tengamos se obtendrán mejores resultados.

Quinta celda de código: pegamos la ruta de best.pt y la ruta de la carpeta imágenes para test

Sexta celda de código: mostramos las imagenes de test



Resultados obtenidos: Podemos ver que el modelo funciona hasta cierto punto, detecto el 100% de los rostros, sin embargo, no obtuvo el mismo éxito al detectar los ojos, para mejorar los resultados debemos agregar más imágenes al dataset ya que para esta prueba solo utilizamos 10 imágenes.

Código fuente: <https://colab.google>

Referencias:

<https://www.youtube.com>

<https://www.youtube.com>

<https://docs.ultralytics.com>

<https://docs.ultralytics.com>

<https://roboflow.com>

Ensamble del vehículo prototipo 29 Enero

El objetivo de este vehículo es simular el funcionamiento de nuestro modelo de detección de fatiga en un vehículo real. El resultado esperado es que en la tercera alerta que se emita el vehículo se detenga por completo para evitar accidentes.





El vehículo cuenta con 3 modos de funcionamiento, seguimiento de línea, eludir obstáculos y control manual controlado mediante control remoto. Se considera utilizar el modo manual para las demostraciones prácticas, sin embargo, esto puede estar sujeto a cambios.

Se removerán las partes que no se vayan a utilizar, como el sensor infrarrojo de seguimiento de línea o el sensor ultrasónico, según el modo de funcionamiento elegido al final.

Código fuente: <https://github.com/Trolhappy>

Este código estará sujeto a cambios y no será utilizado para las pruebas finales.

Investigación sobre el funcionamiento de la librería mediapipe 31 Enero

Ensamble Pironman 5 MAX 1 Febrero

El Pironman 5 MAX es una caja o gabinete de alto rendimiento diseñada para la placa de desarrollo Raspberry Pi 5. Este chasis avanzado amplía sus capacidades para proyectos más exigentes, como NAS, computación de borde, servidores multimedia y tareas de IA ligera.



Se decidió utilizar este gabinete por sus grandes capacidades, como su refrigeración avanzada ideal para mantener una temperatura estable en la raspberry y principalmente para la protección de esta.

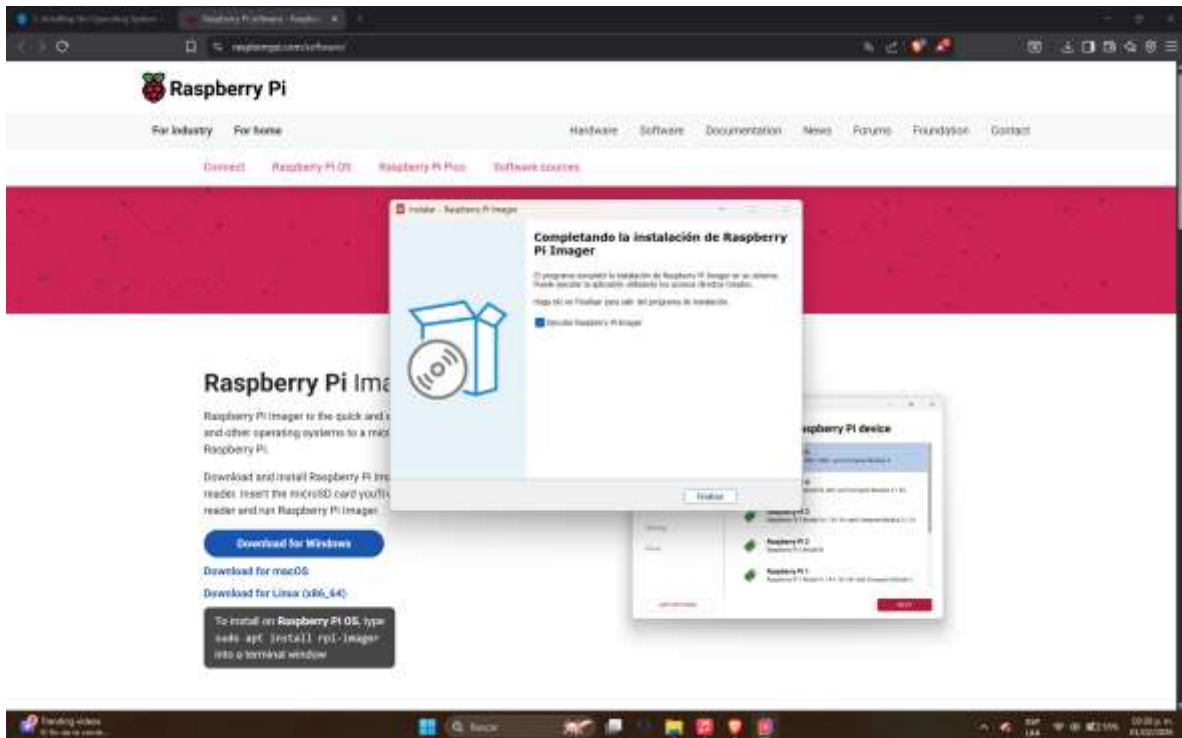


Configuración básica para Raspberry Pi 5 1 Febrero

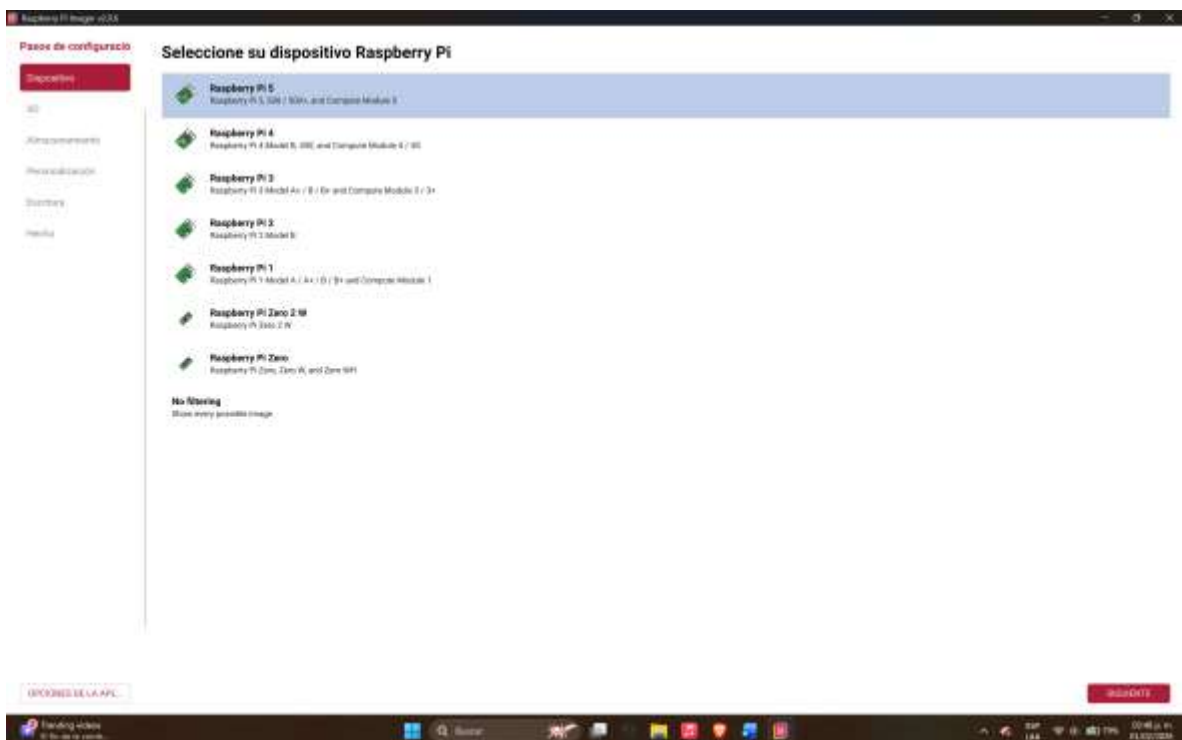
Descargamos el software para instalar un sistema operativo a nuestra Raspberry.

Raspberry Pi Imager: <https://www.raspberrypi.com/software/>

Una vez descargado lo instalamos en nuestra computadora.

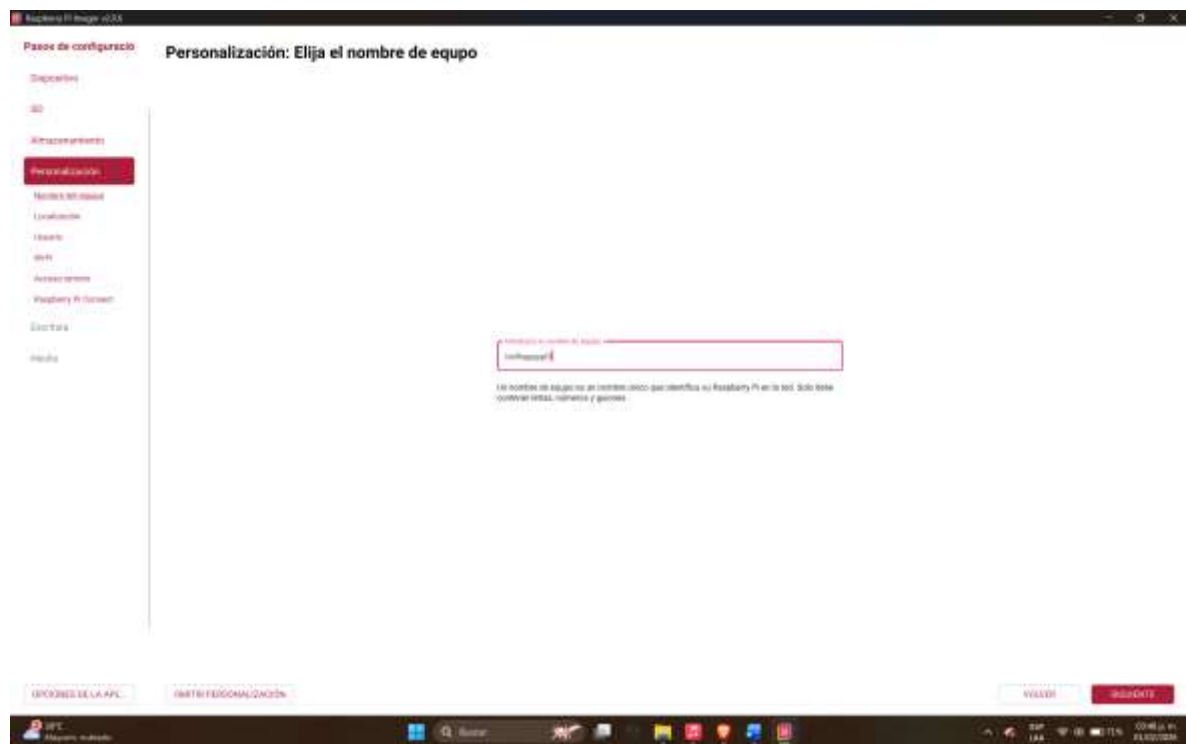


Seleccionamos nuestro dispositivo y damos click en siguiente.

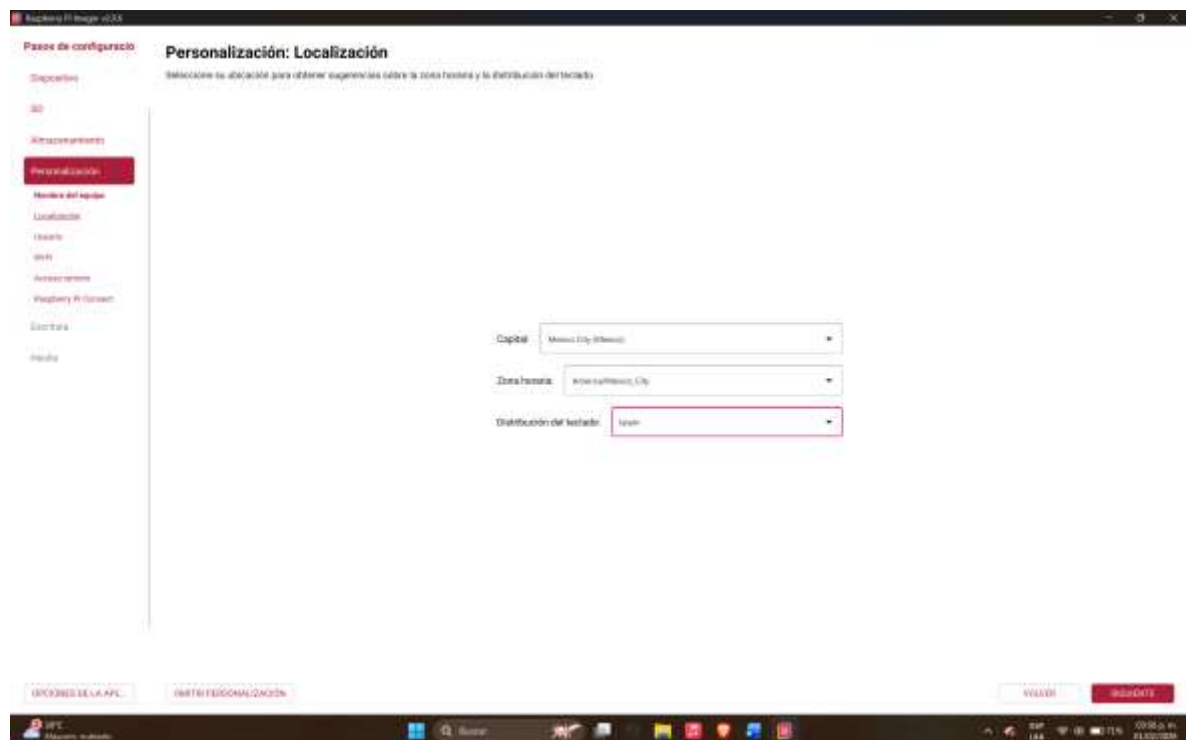


Seleccionamos el sistema operativo y damos click en siguiente.

Introducimos un nombre de identificación.



Seleccionamos nuestra ubicación y tipo de teclado.



Introducimos un usuario y contraseña, importante guardar la contraseña porque se utilizará más adelante.

Raspberry Pi OS

Paseo de configuración

Personalización: Elegir nombre de usuario

Crea una cuenta de usuario para tu Raspberry Pi

Dispositivos

SO

Atascos comunes

Personalización

Reserva del espacio

Localización

Usando

Red

Ahorro de energía

Raspberry Pi Connect

Escritorio

Ayuda

Nombre de usuario: raspberrypi

Contraseña: *****

Confirmar contraseña: *****

El nombre de usuario debe tener un mínimo de 3 caracteres y no puede ser: nombres, palabras clave o guiones.

OPCIONES DE LA APLICACIÓN

OBTENER PERSONALIZACIÓN

VUELTA

SIGUE EN FRENTE

Introducimos los datos de nuestra red para tener conexión a internet.

Raspberry Pi OS

Paseo de configuración

Personalización: Elegir Wi-Fi

Configura la conexión a Internet de tu Raspberry Pi

RED WIRELESS

RED ALBILLO

SSID: PARTICULAR

Contraseña: *****

Confirmar contraseña: *****

☐ Guardar contraseña

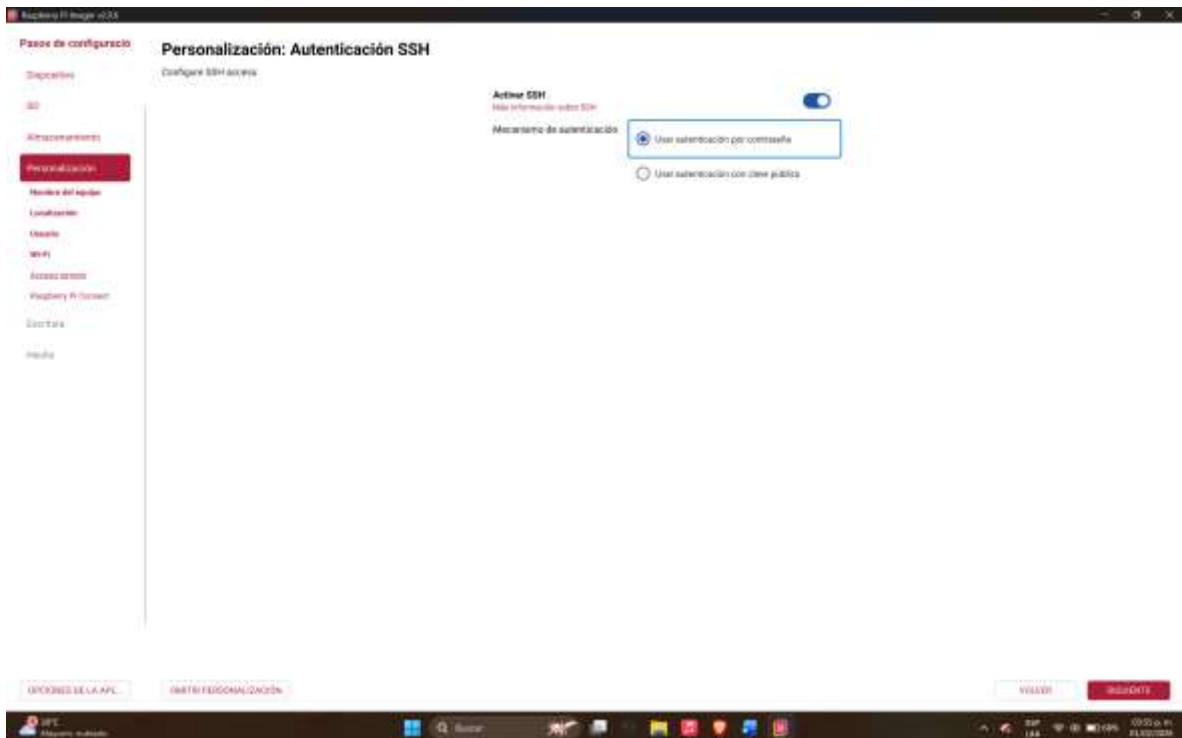
OPCIONES DE LA APLICACIÓN

OBTENER PERSONALIZACIÓN

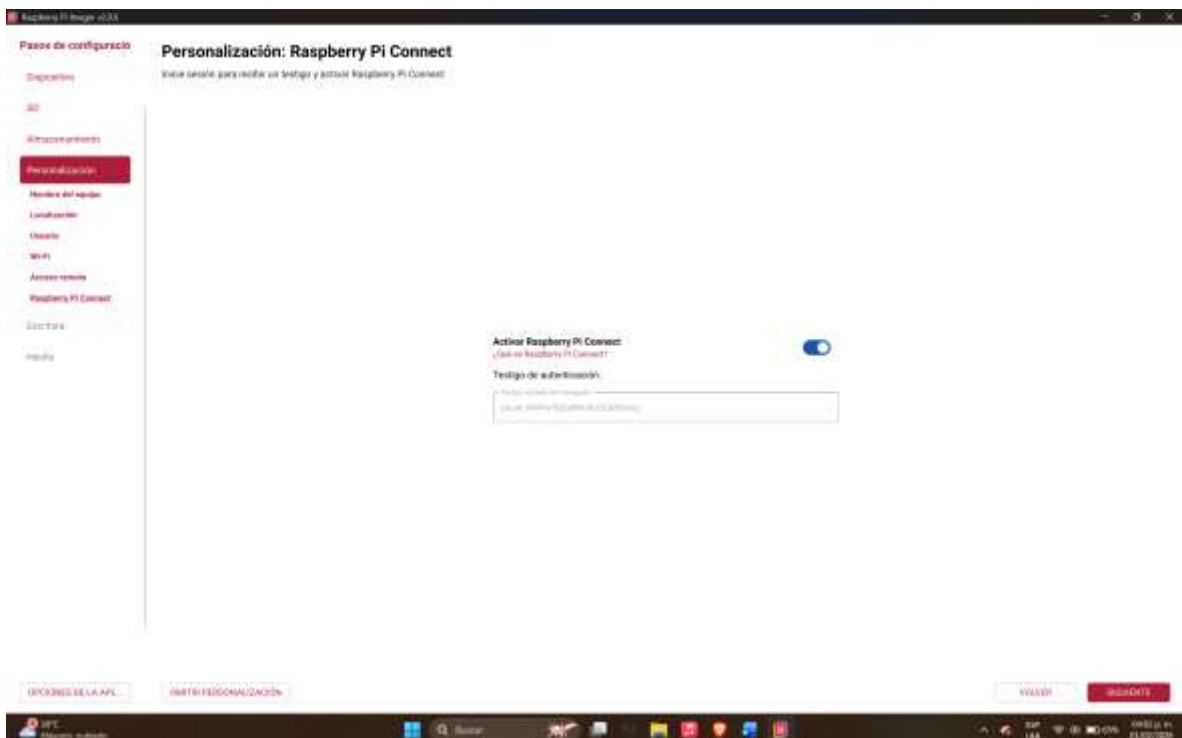
VUELTA

SIGUE EN FRENTE

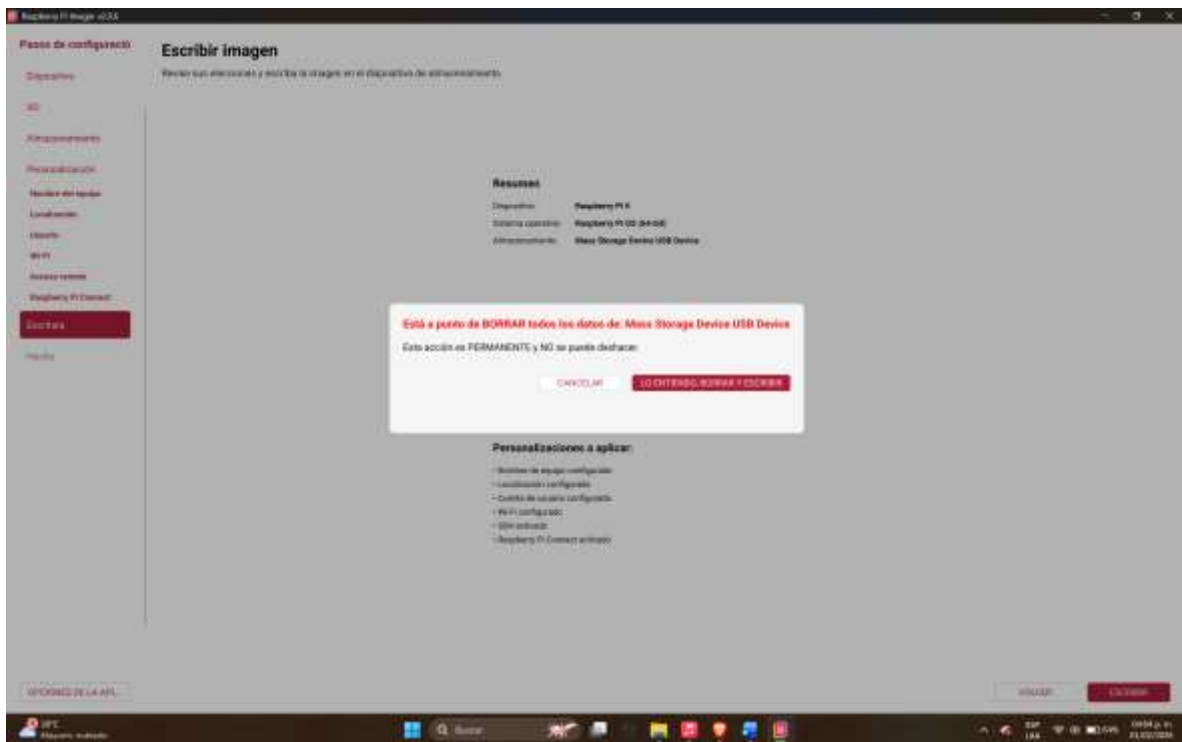
Habilitamos SSH opcional pero recomendado si queremos acceder de forma remota desde nuestra computadora.



Activamos Raspberry Pi Conect (debemos contar con una cuenta), opcional pero útil, podemos acceder al escritorio de la Raspberry desde un navegador web.



Damos en escribir y esperamos a que se instale todo.



Configuración para el hardware Pironman 5 MAX 1 Febrero

He de aclarar que de no contar con el gabinete Pironman 5 MAX estos pasos no son necesarios.

Al encender nuestra Raspberry Pi veremos algo similar a la imagen.



Abrimos la terminal y ejecutamos el siguiente comando.

```
sudo rpi-eeprom-config -e
```

La terminal nos mostrara lo siguiente.

```
[all]
```

```
BOOT_UART=1
```

```
POWER_OFF_ON_HALT=0
```

```
BOOT_ORDER=0xf41
```

Debemos cambiarlo por lo siguiente.

```
BOOT_UART=1
```

```
POWER_OFF_ON_HALT=1
```

```
BOOT_ORDER=0xf41
```

Una vez cambiado presionamos CTRL+X después Y y terminamos con Enter.

Esperamos a que se guarden los cambios y debemos ver algo similar a la siguiente imagen.

Estos comandos sirven para evitar que los ventiladores y la pantalla OLED permanezcan activos al apagarse la Raspberry.


```
git clone https://github.com/sunfouder/pironman5.git
```

```
cd ~/pironman5
```

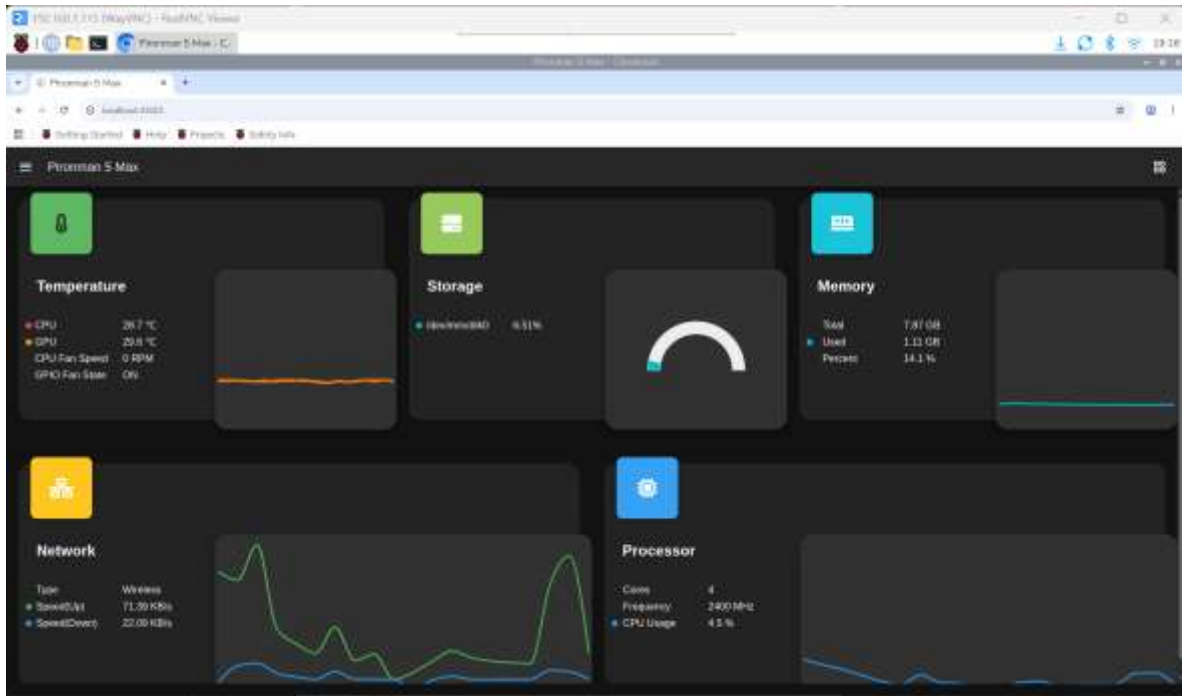
```
sudo python3 install.py
```

Presionamos Y, la Raspberry se reiniciara para aplicar los cambios

[illegible][illegible]

Una vez reiniciada abrimos el navegador y entramos a la siguiente dirección.
localhost.34001

Desde esta pagina podemos monitorear los recursos de nuestra Raspberry.



Acceso a archivos de manera remota Raspberry Pi 1 Febrero

Utilizaremos la aplicación FileZilla, este software nos da acceso a archivos de manera remota y muy fácil de utilizar, además de darnos acceso podemos cargar y descargar archivos lo que nos será útil mas adelante.

FileZilla: <https://filezilla-project.org>

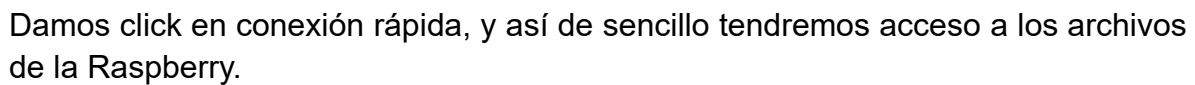
Una vez instalado lo abrimos e introduciremos los siguientes datos.

Servidor: La ip de nuestra Raspberry

Usuario: El usuario que elegimos en Raspberry Pi Imager

Contraseña: La contraseña que elegimos en Raspberry Pi Imager

Puerto: 22



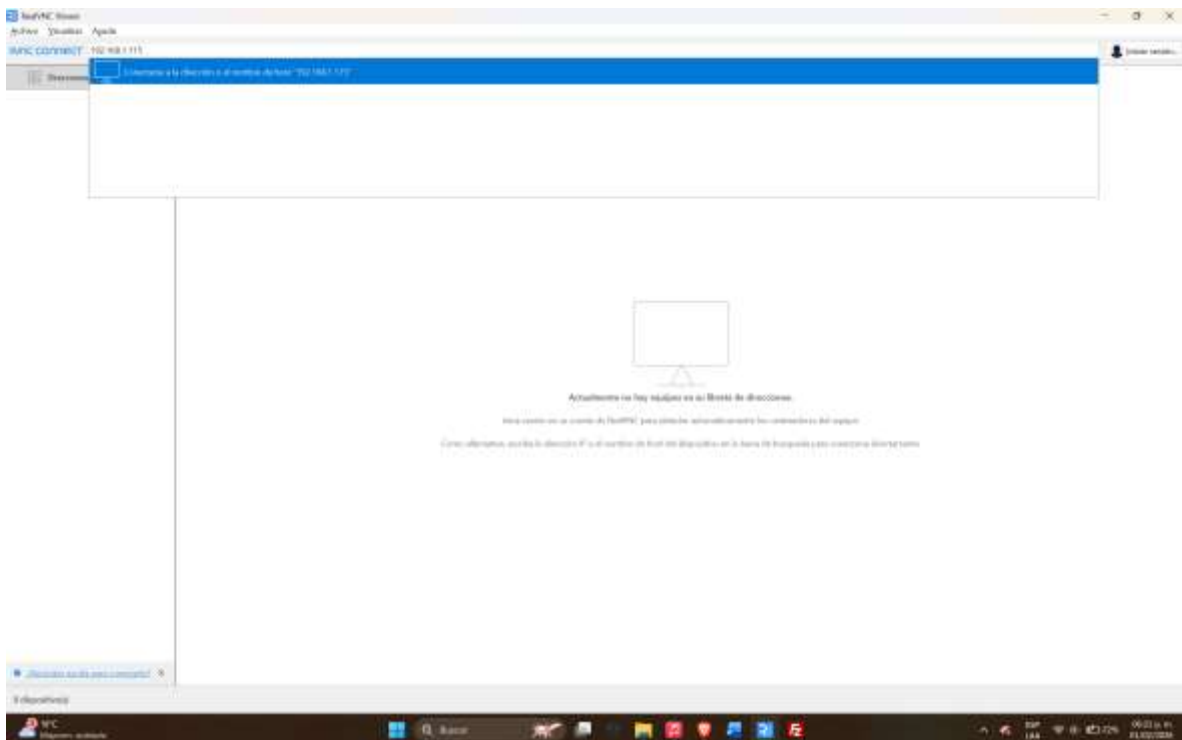
Escritorio remoto Raspberry Pi 1 Febrero

Utilizaremos la aplicación VNC Viewer, el cual es un software de acceso remoto que nos permite no solo visualizar si no también controlar la pantalla de un dispositivo de forma remota. Esto nos facilitara el trabajar para no depender de un monitor o una TV.

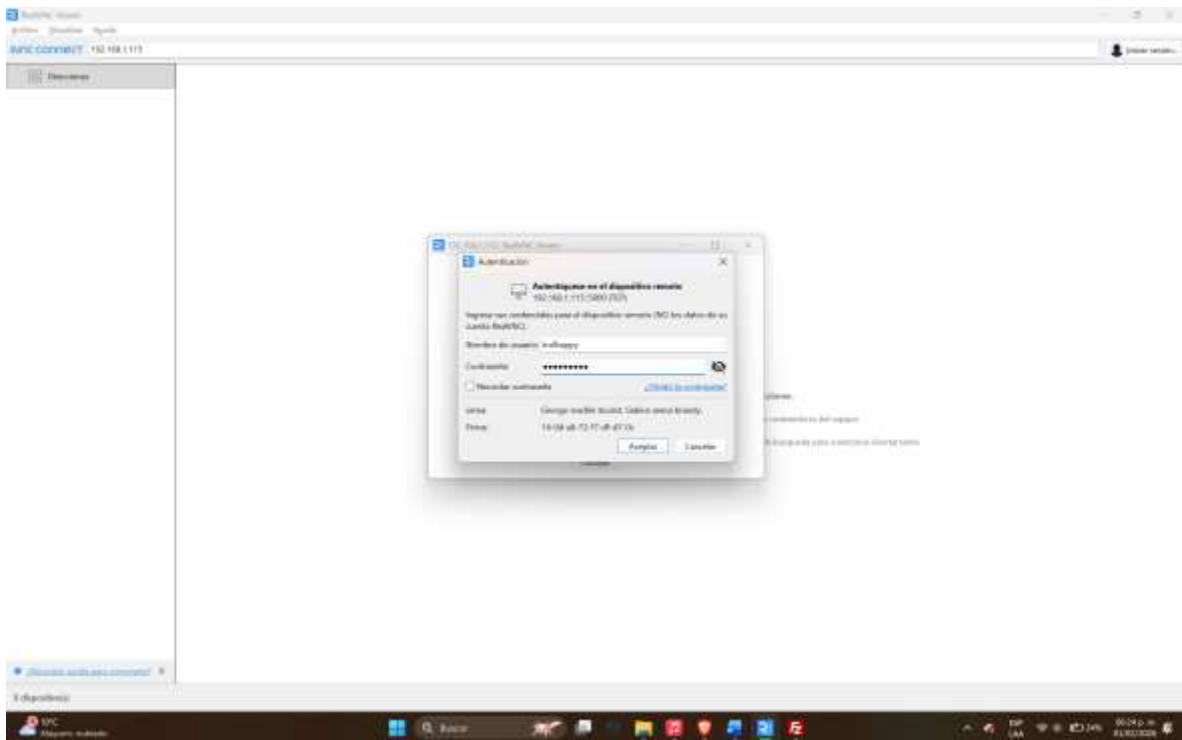
VNC Viewer: <https://www.realvnc.com>

Una vez instalado realizamos los siguientes pasos.

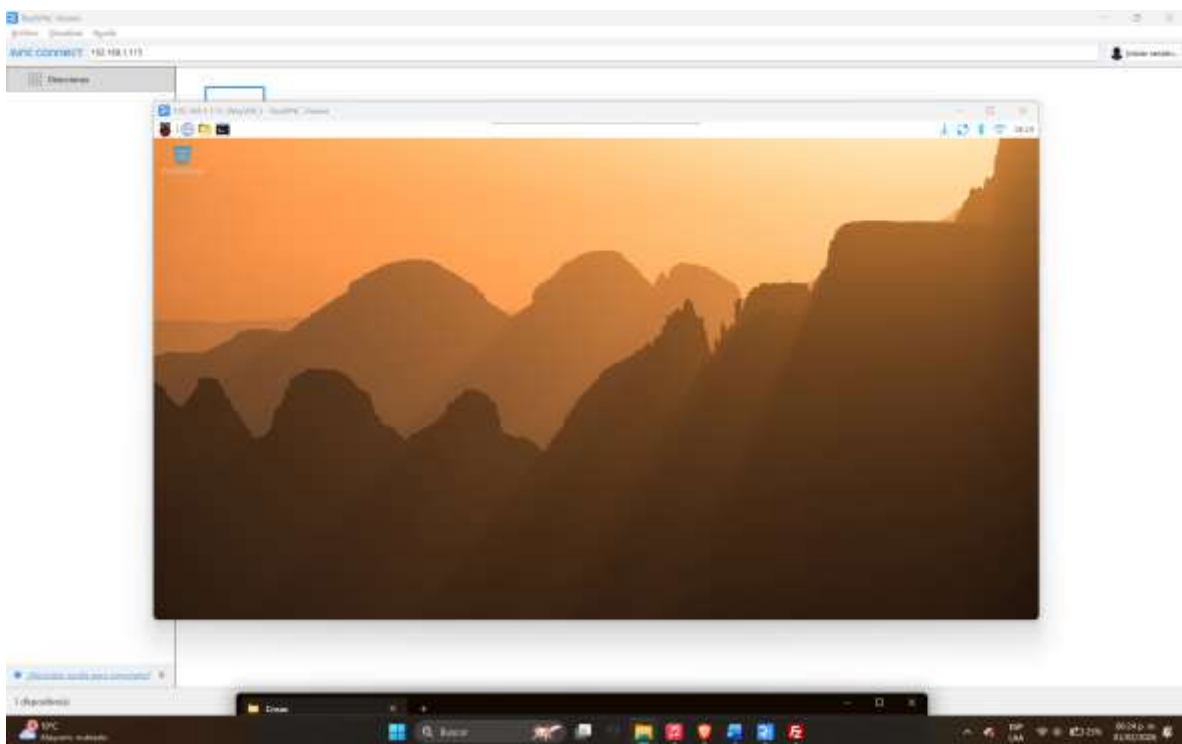
Introducimos la dirección ip de nuestra raspberry y damos click en conectar.



Nos pedirá un usuario y contraseña mismos que configuramos en el Raspberry Pi Imager y damos clic en aceptar.



Y listo así de fácil tendremos acceso a nuestra Raspberry de forma remota.



Instalación de librería Arducam

Abrimos la terminal y ejecutamos los siguientes comandos

Descargue los scripts bash

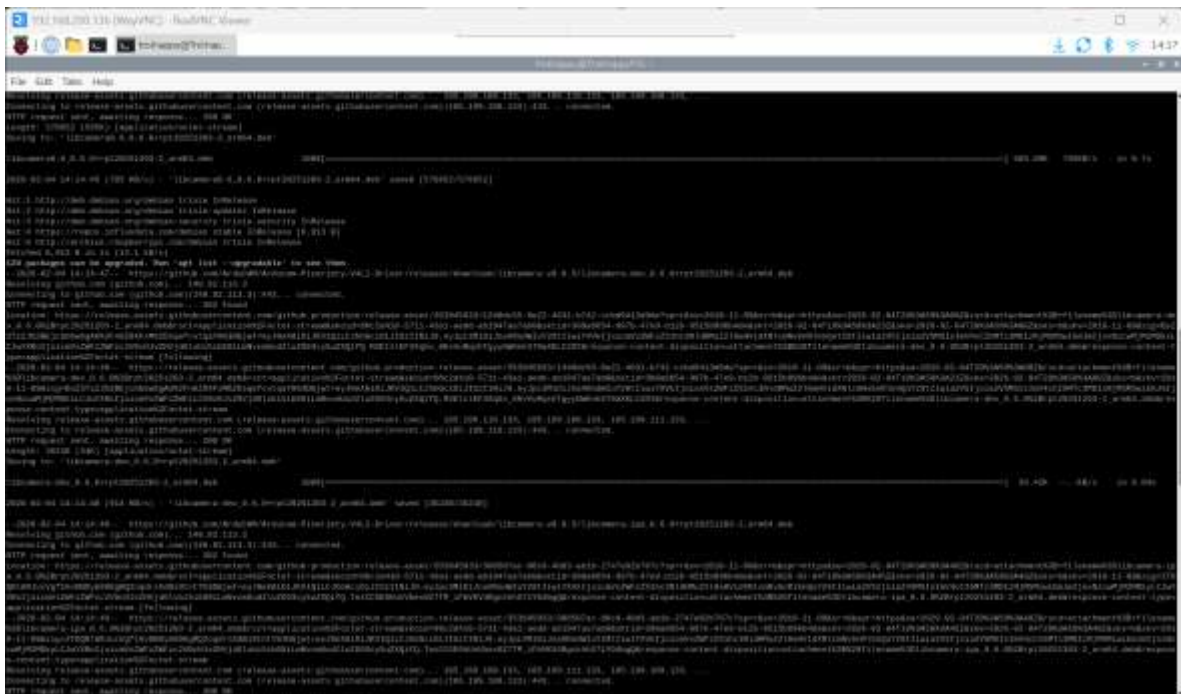
```
wget -O install_pivariety_pkgs.sh https://github.com/ArduCAM/Arducam-Pivariety-V4L2-Driver/releases/download/install_script/install_pivariety_pkgs.sh
```

```
chmod +x install_pivariety_pkgs.sh
```

[illegible]

Instalamos libcamera

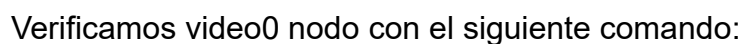
```
./install_pivariety_pkgs.sh -p libcamera dev
```



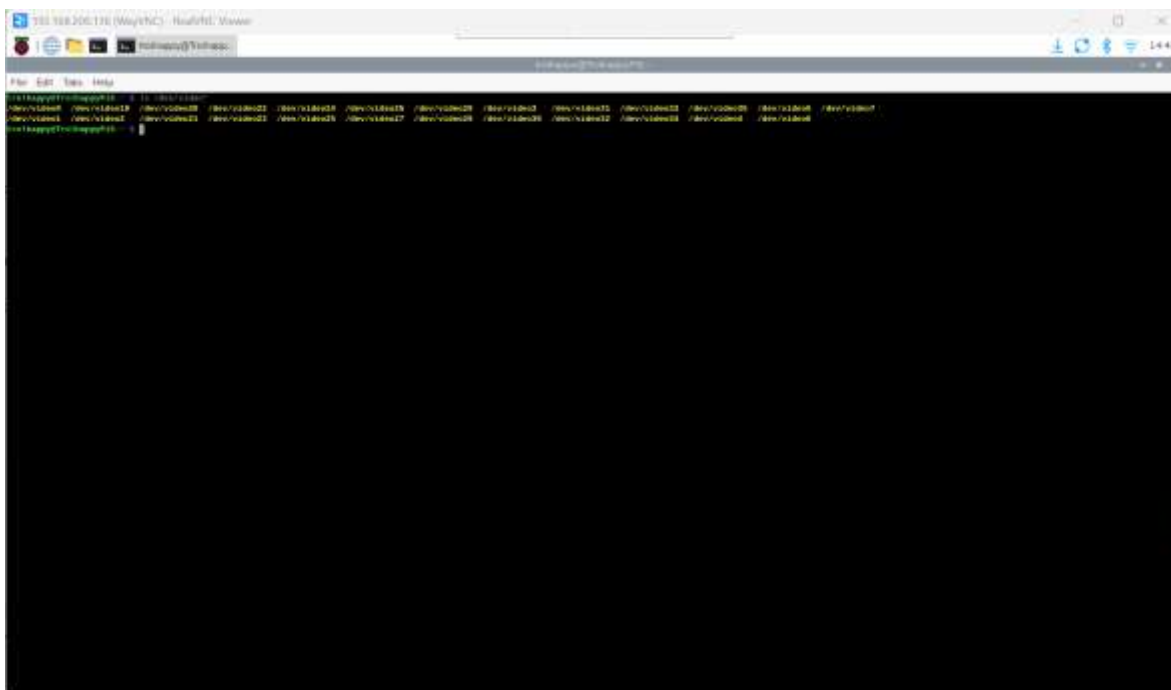
Instalamos de libcamera-apps




```
dmesg | grep imx415
```

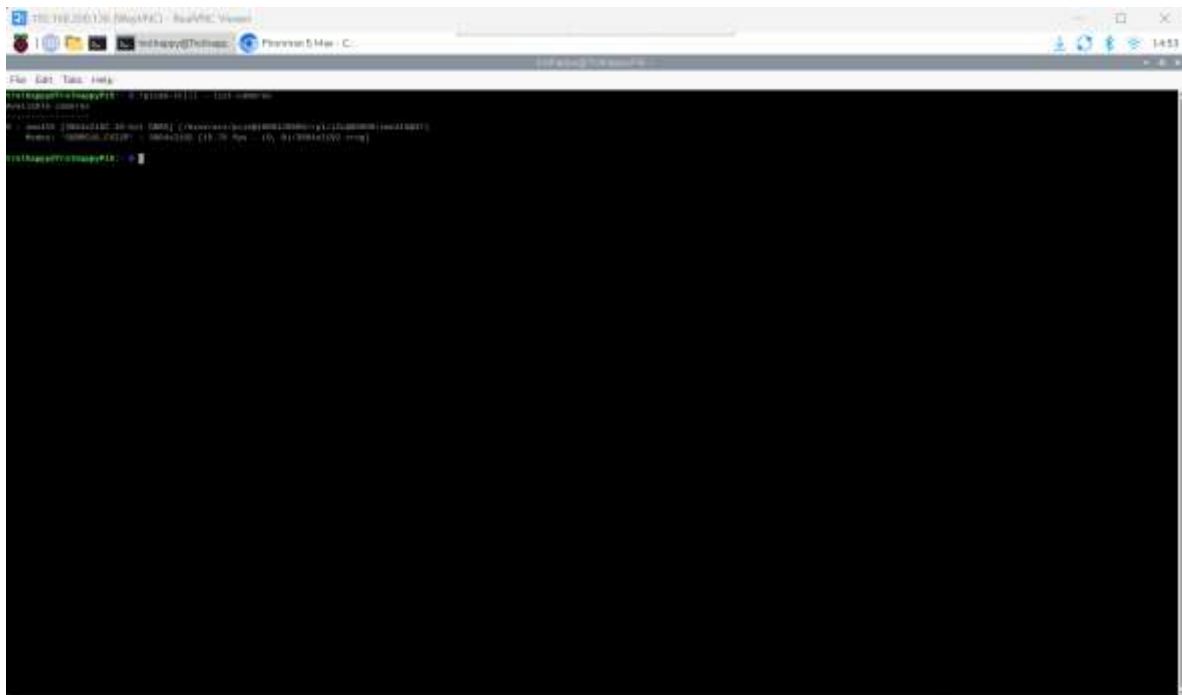


```
ls /dev/video*
```



Verificamos la resolución y velocidad admitidas por la cámara con el siguiente comando:

`rpikam-still --list-camera`



```
Y19279E200126 [RaspFRC] - RaspFRC Viewer
root@raspberrypi:~# rpikam-still --list-camera
rpikam-still: listing cameras...
Found 1 camera(s):
0: model: [1000000000-0000] [0000] (0x0000000000000000) [0x0000000000000000]
  Name: [00000000-0000] [10-00-00] [0x0000000000000000]
```


Librerías necesarias para ejecutar nuestro código

Ejecutaremos los siguientes comandos desde terminal

COMANDO	FUNCION
<code>sudo apt update sudo apt install -y build-essential libssl-dev zlib1g-dev \ libbz2-dev libreadline-dev libsqlite3-dev curl \ libncursesw5-dev xz-utils tk-dev libxml2-dev libxmlsec1-dev libffi-dev liblzma-dev</code>	Instalar dependencias para compilar Python
<code>curl https://pyenv.run bash</code>	Instalar pyenv
<code>export PATH="\$HOME/.pyenv/bin:\$PATH"</code>	
<code>eval "\$(pyenv init -)"</code>	
<code>eval "\$(pyenv virtualenv-init -)"</code>	
<code>source ~/.bashrc</code>	Reconoce los comandos anteriores sin reiniciar la Raspberry
<code>pyenv install 3.11.9</code>	Instalar la versión estable de Python para MediaPipe
<code>mkdir -p ~/detector_de_fatiga</code>	Crear una carpeta
<code>cd ~/detector_de_fatiga</code>	Entrar en la carpeta
<code>python3 -m venv env</code>	Crea el entorno virtual
<code>source env/bin/activate</code>	Activa el entorno virtual
<code>pip install mediapipe opencv-python pyserial</code>	Instalar las librerías

Cada vez que apaguemos y encendamos la raspberry debemos volver a entrar a la carpeta y activar el entorno virtual con los comandos anteriores.

`v4l2-ctl --list-devices`

